Caching Best Practices

The Senior Developer's Guide to ASP.NET Core Performance

1. Universal Caching Principles

Event-Driven Invalidation (Preventing Stale Data)

Never rely solely on time-based expiration (Absolute/Sliding) for data that changes. Always explicitly call `Remove()` or `RemoveAsync()` inside your Create, Update, and Delete methods immediately after saving to the database.

Pass Cancellation Tokens

When handling a Cache Miss, your callback usually queries the database. Always pass a `CancellationToken` to prevent Thread Pool Starvation if the SQL server locks up.

2. L1 Cache: IMemoryCache Guidelines

The Immutability Trap

Data returned from `IMemoryCache` is a direct pointer to the C# object in RAM. If you mutate its properties, you change the data for all subsequent requests. Always treat cached L1 objects as strictly read-only.

The Memory Leak Danger

By default, `IMemoryCache` has infinite size. You must configure a global `SizeLimit` in `Program.cs` and set `entry.Size = 1;` on every cache entry to prevent OutOfMemory exceptions.

3. L2 Cache: IDistributedCache (Redis) Guidelines

> **The Serialization Tax**
>
> You cannot send raw C# objects over a network. You must explicitly serialize to JSON before `SetStringAsync` and deserialize after `GetStringAsync`. Ensure your objects do not contain EF Core circular navigation properties.

> **Graceful Degradation (Fault Tolerance)**
>
> Do not let a Redis outage crash your API. Wrap your Redis calls in a `try/catch` block. If Redis throws an exception, log the error and either fetch directly from SQL (for reads) or save directly to SQL (for writes).

4. The Modern Standard: HybridCache

In modern .NET applications (.NET 9+), prefer **HybridCache** over manual L1/L2 orchestration. It automatically combines local RAM speed with Distributed Cache durability.

> **Cache Stampede Protection**
>
> Use HybridCache to prevent the "Thundering Herd" problem. If a popular key expires, HybridCache automatically locks the execution thread. It allows only one request to hit the database, while concurrent requests wait milliseconds for the cache to populate.

5. L3 Cache: SQL Server as a Cache

When Redis is too expensive or prohibited by infrastructure policies, use the `DistributedSqlServerCache`.

- **When to use:** For massive datasets that exceed affordable RAM limits, or when you require cache persistence that survives a full datacenter reboot.
- **Architecture Strategy:** Use the Decorator Pattern to wrap both Redis (L2) and SQL Server (L3) into a single `IDistributedCache` implementation, feeding that composite cache into `HybridCache`.